

Review of the BARK vs. SKQD implementation

Heisenberg study (`EqualNumberOfBitstrings.py`), second version

Code review for E. Rosanowski

23 September 2026

Abstract

This version replaces the first review. It covers the code as it is now, after C1 (energy feedback in BARK), SKQD’s patience, the new tuning objective and grid, the filters for trivial and quasi-degenerate ground states, the median plots, and the stored SKQD hyperparameters. Every remaining issue is sorted into one of three classes. **Blocking** means a referee will find it and the paper cannot be accepted as long as it stands. **Likely raised** means a careful referee will probably ask, and it is cheap to pre-empt. **Taste** means no action is needed. Five issues were classed as blocking. One of them (B3) is now addressed in the code, and two of the remaining four are about how the result is *stated*, not about the code. All numbers from the results file refer to the data produced *before* the fixes, so they show the size of the effects but must be recomputed after the re-run.

Verdict in one paragraph

The numerics are sound, and the four fixes remove the ways in which SKQD was handicapped and BARK’s cost was hidden. What can still stop the paper:

- B1 the claim “for all parameters” is likely false in the data;
- B2 the claim about classical cost needs a complexity argument written into the paper;
- B3 the plotted mean advantage is set mostly by the arbitrary 10^{-16} floor on the infidelity.
Addressed: median plots now exist, and the paper must quote those;
- B4 `BARK.sweep` crashes, and still uses exact diagonalization inside the selection. This is blocking only if the fixed-target study is part of the paper;
- B5 every result has to be regenerated with the current code.

1 What the study does now

Hamiltonians. Uniform XXZ model with open boundaries, in the Pauli convention (`spin=1`):

$$H = J \sum_{\langle ij \rangle} (X_i X_j + Y_i Y_j + \Delta Z_i Z_j) - B_z \sum_i Z_i,$$

on a chain or the most-square open rectangle. The parameters are $N \in \{6, 8, 10, 12\}$ and $\Delta \in \{0, 0.5, 1, 10, 100\}$, with 15 draws of $(J, B_z) \sim U[-1, 1]^2$, which makes 600 cells. Up to six initial basis states are drawn uniformly among those with non-zero ground-state overlap.

Protocols.

- **BARK** (`full_bark_run`): the candidates are the rows of H at the last added state. They are scored with Johann’s 2×2 estimate, which uses the energy of Johann’s own running approximation (C1 fix). The scores are kept in a heap, and the best state is added. The exact diagonalization at every step now only serves as the metric.

- **SKQD** (`full_skqd_run`): $|\psi_k\rangle = U(t)^k|s_0\rangle$, exact. n shots are drawn per step. The run stops after more than five consecutive steps with no new bitstring. (t, n) is chosen once per run on a 7×5 grid, $t \in \{0.02, \dots, 10\}$ and $n \in \{10, \dots, 1000\}$, with a parabolic refinement. The choice minimizes the mean infidelity over the whole budget grid and is then evaluated on 3 fresh repeats.
- **Simplified BARK**: the same loop as BARK, scored by $-|H_{s_{\text{last}}, s}|$.
- **Ceiling**: the summed probability of the K most probable ground-state components. It is an upper bound for any method.

Output and plots. Every row now also stores the gap $E_1 - E_0$ of its cell (`Gap`) and, on SKQD rows, the tuned hyperparameters (`T`, `N_Shots`). Before plotting, two filters remove cells: `drop_trivial_ground_states` (ground state is a single basis state; 259 of 600 in the current data) and `drop_small_gaps` (gap below `MIN_GAP`= 10^{-4} ; 53 cells in the current data). Each cluster folder holds the curves and ratios three times: mean with standard deviation, mean with standard error, and median with interquartile range (`_median`), each also as a `_zoom` version that stops in front of the drop of the ceiling.

2 Status of all issues

ID	Issue	Status
C1	BARK ranked with the ED energy (and needed ED every step)	fixed (and the resulting crash, too)
C2	SKQD stopped at the first empty batch	fixed (patience 5, both functions)
C3	SKQD tuned only at K_{max}	fixed (whole trajectory, wider grid)
C4	“all parameters” contradicted	open, see B1
C5	trivial ground states	fixed (filtered in the plots)
C6	budgets past the symmetry sector	taste
M1	quasi-degenerate ground states	fixed (gap filter, see L1)
M2	stale scores in BARK’s heap	taste, see T2
M3	Simplified BARK breaks ties by bitstring index	open, see L2
M4	no cost measured, dense 2^N vectors	open, see B2 and T3
M5	time grid not scaled with $\ H\ $	withdrawn (tested: no gain), see L4
m1–m8	minor	see Section 5
B3	mean of $\log(1 - F)$ set by the floor	fixed in the code (median plots); quote medians
B4	<code>BARK.sweep</code> broken	open, only if the fixed-target study is used
L1	quasi-degenerate ground states	fixed (<code>Gap</code> column, <code>MIN_GAP</code> = 10^{-4})
L4	hyperparameters not recorded	fixed (<code>T</code> , <code>N_Shots</code>); grid edge to check after the re-run

The following were checked and are correct: the projected blocks (`GrowingProjection`), SKQD sampling and time evolution, Johann’s formula, `fidelity_at_budget`, the ceiling (no fidelity in the CSV exceeds it), the LOBPCG warm start (identical pools to `eigh`), the seeding, and both new patience loops (pool sizes strictly increasing; identical results with and without `budgets`). The new columns, both filters and the median plots were tested on one fresh cell and on the old $N = 6$ data.

3 Blocking

B1 [blocking] “BARK outperforms SKQD for all parameters”

Why it blocks. This is a universal statement. A single counterexample falsifies it, and the code and data will be public. In the current data, 5–8% of the non-trivial runs at every N lose to SKQD by $\Delta F > 0.05$ at some budget below the sector size:

N	6	8	10	12
runs losing by $\Delta F > 0.05$	6.9%	4.9%	8.3%	4.8%
runs losing by $\Delta F > 0.1$	4.6%	3.7%	5.2%	2.9%

The share is flat in N , so the problem does not grow with system size. It is concentrated at $\Delta = 0$ (12–19% of runs). The SKQD fixes will make SKQD stronger. On the $N = 10$, $\Delta = 0.5$ test cell, SKQD with the new tuning reaches $F = 0.48$ and 0.82 at $K = 48$ and 92 , against 0.27 and 0.58 for BARK.

The three repeats per run are too few to call a single loss significant. As discussed, a per-run test is not possible, but it is also not needed. The shares above are statistics over about 500 runs per N .

What to do. After the re-run, recompute this table. If losses remain, state the claim as a fraction: “BARK reaches a higher fidelity than SKQD in $X\%$ of runs, and on average by $\dots$; the share does not grow with N ”. That statement is publishable. A universal one is not.

B2 [blocking] The cost claim needs an explicit argument

Why it blocks. Equal classical cost is half of the paper’s thesis, and the study records no cost at all. That is acceptable if the paper contains a complexity argument, but not if the claim is only asserted. After C1, the argument holds. Let d be the number of non-zero entries per row of H ($d \approx$ number of bonds $+1$), and assume vectors are stored on the pool only.

- *BARK selection*: per step, d candidates. Each needs $\langle s|H|v \rangle$ in $O(d)$, plus a heap operation in $O(\log K)$. In total this is $O(Kd^2 + Kd \log K)$, with no diagonalization.
- *SKQD classical part*: deduplication of the shots, in $O(\text{shots})$.
- *Final estimate*, identical for both methods: ED on the K -state pool (dense $O(K^3)$, or sparse Lanczos $O(Kdm)$), or Johann’s method in $O(Kd^2)$.

So BARK’s overhead is at most linear in K up to a logarithm, and never above the final estimate both methods share. This is the argument for why BARK does not re-score its frontier (T2). Also state that the exact diagonalization at every step in `full_bark_run` is a measurement device of the study, not part of the algorithm. Consequently, wall times measured with this code must not be quoted as BARK’s cost (see also T3).

B3 [blocking], now fixed in the code: the plotted mean advantage depends on the infidelity floor

Why it blocks. The curves and ratio plots average $\log_{10} \max(1 - F, 10^{-16})$. BARK reaches $F = 1$ to machine precision much more often than SKQD, because a pool that covers the ground-state support gives an exact result. Those rows sit at an arbitrary constant. In the current data, excluding trivial cells:

floor	BARK rows at floor	SKQD rows at floor	mean / median $\log_{10}(\text{BARK/SKQD})$
10^{-16}	13%	4%	$-2.71 / -0.29$
10^{-10}	22%	5%	$-1.77 / -0.29$
10^{-6}	23%	5%	$-1.09 / -0.29$

The mean advantage changes by a factor of 40 with a choice that has no physical meaning. The median is stable. A referee who asks how the floor was chosen gets no satisfying answer, and “2.7 orders of magnitude” will not survive review. The direction of the result (BARK better) is robust.

Status. Done: `aggregate(..., error="median")` gives the median of $\log_{10}(1 - F)$ with the interquartile range as band, and every cluster now has `curves_median.pdf` and `ratio_median.pdf` (plus `_zoom`). The mean-based figures are unchanged.

What remains. Use the median figures in the paper and quote medians in the text. The median is not immune to the floor either: once more than half of the runs in a cluster reach $F = 1$

exactly, it drops to the floor as well. In the data this happens beyond the size of the symmetry sector. The `_zoom` versions stop before that drop, so they are the ones to show. This part is presentation, not code.

B4 [blocking] (only if the fixed-target study is in the paper) BARK.sweep is broken

Since commit 4afda6a02, `rank_states` returns three arrays. `BARK.sweep` (l. 238) still treats its result as one array of scores and fails with `TypeError: only 0-dimensional arrays can be converted to Python scalars`. `ClusterStudy.py` calls it, so the fixed-target study cannot run with the current code. Any fixed-target results on disk come from an older BARK. Moreover, `sweep` ranks with the ED vector and ED energy, which is the C1 problem again. If that study is to appear, `sweep` must use the same selection as `full_bark_run` (Johann vector and `new_energy`), and the study must be re-run. If it does not appear, this has no consequence for the paper.

B5 [blocking] All results must be regenerated

The CSV and all figures were produced before C1, C2, C3 and the new grid. Each of these changes the curves; the test cell above changed qualitatively. Nothing from the current data may go into the paper. After the re-run, recheck B1 and B3 and the grid edge (L4) before writing.

4 Likely raised by a referee

L1 [likely raised] Quasi-degenerate ground states at large Δ

53 cells have a gap $E_1 - E_0 < 10^{-4}$ and 14 a gap $< 10^{-8}$ (down to 6×10^{-11}), all at $\Delta \in \{10, 100\}$ with $J > 0$. The two Néel-like states are split only by high-order tunnelling. The fidelity against one eigenvector is then ill-conditioned, and a pool containing one Néel sector is capped near $F \approx 0.5$ although its energy is essentially exact. Someone working on XXZ models will notice this in the $\Delta = 10, 100$ panels.

Status: **fixed.** `run_cell` stores the gap per cell (`Gap`, from the eigendecomposition that is computed anyway), and `drop_small_gaps` removes cells with a gap below `MIN_GAP` = 10^{-4} from the plots. The gaps have no natural break, so the threshold is a choice. On the current data it would remove 53 cells: 45 at $\Delta = 100$ and 8 at $\Delta = 10$. State the threshold and the count in the paper. Reporting the energy error $|E_K - E_0|/|E_0|$ next to F remains an optional extra.

L2 [likely raised] Simplified BARK is decided by the bitstring index

In the uniform XXZ model every off-diagonal element has the modulus $2|J|$, so all candidates score the same and the heap takes the smallest integer label. The curve depends on how the qubits are numbered. If it appears in the paper, a referee who understands the method will object. Drop it from the Heisenberg figures, or give it a tie-break (e.g. the diagonal H_{ss}) and say so.

L3 [likely raised] SKQD may receive fewer than K states at budget K

SKQD is read at the last pool size $\leq K$, so the final batch before the budget is discarded entirely. With the new grid of up to 1000 shots per step, this can be hundreds of states, while BARK always gets exactly K . The optimizer sees the same accounting and will avoid large n where it hurts, which softens the effect. Still, a referee can call the comparison unfair. Either state it explicitly, or read SKQD at exactly K . That requires two changes: keep the pool in draw order (`GrowingProjection.extend` sorts via `np.unique`), and diagonalize `prefix_block(K)` at each budget.

L4 [likely raised] Hyperparameters: recorded; the grid edge remains to be checked

Status: fixed. The SKQD rows now carry the chosen (t, n) in the columns `T` and `N_Shots`, so a referee’s first question, “what did you use for SKQD?”, can be answered, and the distribution of the choices can be reported.

What remains. On the test cell the optimum was $t = 0.02$, the smallest value on the grid. If the optimum is at an edge in many runs after the re-run, a referee will say SKQD was not tuned, and the grid should be extended in that direction. Scaling t with $\|H\|$ was tested and brings no gain, because the spreading of amplitude is set by the flip-flop terms $\sim |J|$ and not by the diagonal.

L5 [likely raised] Scope: initial states and system size

The initial states are uniform over the support, so their overlaps are mostly tiny, while SKQD is used with a good reference state (Néel or mean field) in practice. The overlap-binned figures already cover this; show the high-overlap bin in the paper. $N \leq 12$ will draw the question of whether the conclusion survives at scale. The flat loss share in B1 and the cost argument in B2 are your answer, so present both.

5 Taste, no action needed

T1 [taste] Axis normalisation

$K/2^N$ instead of the sector dimension, and budgets beyond the sector. The curves remain correct.

T2 [taste] Stale scores in the heap

Re-scoring the frontier would improve BARK (tested: $F = 0.31 \rightarrow 0.97$ at $K = 7$ on one cell), but costs $O(K^2 d)$ and breaks cost parity (B2). Scoring once at discovery is the right choice for this paper. Lazy re-scoring of only the popped top element would keep parity, if you ever want more.

T3 [taste] Dense 2^N vectors in full_bark_run

(johanns_approximation, new_state) The implementation costs $O(K 2^N)$ although the algorithm does not need it. This is irrelevant for $N \leq 12$ and for an analytic cost argument. It only matters if wall times are reported.

T4 [taste] float(beta[0])

drops $\text{Im } \beta$; exact for the real XXZ matrices used here.

T5 [taste] LOBPCG acceptance

checks only the residual; no wrong eigenpair was observed.

T6 [taste] Patience of 5

Arbitrary, but reasonable. State the value.

T7 [taste] Tuning read-out

Budgets below SKQD’s first recorded pool are read as $F = 0$ in the optimizer, but as the initial overlap in the evaluation. The overlap is tiny, so the effect is negligible.

T8 [taste] Conventions

State the Pauli convention, and that “2D” for $N \leq 10$ means $2 \times L$ ladders.

T9 [taste] Submit script

Stale comment on `NUM_INITIAL_STATES`; the default for `--num-hamiltonians` differs from the script (20 vs. 15); unquoted `#{STUDY_ARGS[*]}` in `--wrap`.

6 Before the re-run

1. Done: the columns `T`, `N_Shots` and `Gap` (L4, L1), the gap filter (L1) and the median plots (B3).
2. Decide on L3 (exact- K read-out), since it changes what is stored.

3. If the fixed-target study stays in: fix `BARK.sweep` (B4).
4. Point the re-run at fresh output. `run_job` skips every shard file that already exists, and `bitstring_shards_heisenberg/` still holds the old ones. Use a new `SHARD_DIR`, `OUTPUT` and `OUTPUT_ROOT`, or pass `EXTRA_ARGS="--overwrite"`. Old `.partial` files are rejected automatically, because they lack the new columns.
5. After the re-run: recompute the loss-share table (B1), use the median `_zoom` figures (B3), and check how often (t, n) sits at the edge of the grid (L4).
6. In the text: phrase B1 as a fraction, write out the complexity argument (B2), and state the conventions and the patience.